204252 Data Structures and Analysis

Chapter 10: Sorting (Part 2)

Lecturer

August 24, 2026

Computer Science Department

Outline

Quicksort

Heapsort

Non-Comparison Sorts (Linear Time)

Quicksort

Quicksort: Core Idea

Concept:

- A recursive, divide-and-conquer algorithm like Mergesort, but sorts **in-place**.
- **Divide**: Pick a pivot element v. Partition the remaining elements of the array S into:

$$S_1 = \{x \in S \setminus \{v\} \mid x \leq v\}$$
$$S_2 = \{x \in S \setminus \{v\} \mid x \geq v\}$$

- **Conquer**: Recursively sort S_1 and S_2.
- **Combine**: Concatenate the sorted S_1, followed by v, followed by the sorted S_2. Since partitioning is done in-place, no extra combination step is required!

Pivot Selection Strategies

Choosing a good pivot v is critical for partitioning balance:

- **First Element (The Wrong Way)**:
  - If input is already sorted or in reverse order, the pivot will be the absolute minimum or maximum.
  - Result: One partition is empty, the other has $N - 1$ elements.
  - Worst-case recurrence: $T(N) = T(N-1) + \mathcal{O}(N) \implies \Theta(N^2)$ run time!
- **Randomly Picked Pivot (A Safe Maneuver)**:
  - Safe against sorted inputs, but random number generation is expensive.
- **Median-of-Three Partitioning**:
  - Estimate the median of the array by taking the median of the first, center, and last elements:

$$v = \text{median}(a[\text{left}], a[\text{center}], a[\text{right}])$$

  - Eliminates the worst-case for sorted inputs and reduces comparisons by $\approx 14\%$.

Median-of-Three Selection C++ Code

```
template <typename Comparable>
const Comparable & median3( vector<Comparable> & a, int left, int right ) {
    int center = ( left + right ) / 2;
    if( a[ center ] < a[ left ] )
        std::swap( a[ left ], a[ center ] );
    if( a[ right ] < a[ left ] )
        std::swap( a[ left ], a[ right ] );
    if( a[ right ] < a[ center ] )
        std::swap( a[ center ], a[ right ] );

    // Place pivot at position right - 1 (hide it)
    std::swap( a[ center ], a[ right - 1 ] );
    return a[ right - 1 ];
}
```

Side effects of Median-of-Three Selection

Side effects of ‘median3‘:

- Sorts a[left], a[center], and a[right] in place.
- Places the pivot at $a[\text{right} - 1]$.
- a[left] acts as a sentinel for j, and a[right] is already $\geq$ pivot.

In-Place Partitioning Strategy

Goal: Group elements $\leq$ pivot on the left, and elements $\geq$ pivot on the right.

1. Initialize pointers $i = \text{left}$ and $j = \text{right} - 1$.
2. Outer loop:
   - Increment i while $a[i] <$ pivot.
   - Decrement j while $a[j] >$ pivot.
   - If $i < j$, swap $a[i]$ and $a[j]$, then repeat.
   - Else, break.
3. Restore pivot: Swap $a[i]$ with $a[\text{right} - 1]$ (the pivot cell).

Critical detail - Equal Keys:

- If $a[i] ==$ pivot and $a[j] ==$ pivot, we **stop** both pointers and swap them.
- Although this does unnecessary swaps, it prevents pointers from running past and guarantees balanced partitions on arrays filled with duplicate keys.

Small Subarrays: Optimization Cutoff

- For very small arrays ($N \leq 10$), Quicksort is slower than simple sorting algorithms (like Insertion Sort) due to recursive function call overhead.
- **Optimization**:
  - Do not recursively quicksort subarrays of size ≤ 10.
  - Instead, invoke **Insertion Sort** directly on those small segments.

Cutoff Condition
If the range size is less than 10:

$$\text{right} - \text{left} + 1 < 10 \implies \text{left} + 9 \geq \text{right}$$

Quicksort: C++ Implementation

```
1  template <typename Comparable>
2  void quicksort( vector<Comparable> & a, int left, int right ) {
3      if( left + 10 <= right ) { // Size >= 10: use quicksort
4          const Comparable & pivot = median3( a, left, right );
5          int i = left, j = right - 1;
6          for( ; ; ) {
7              while( a[ ++i ] < pivot ) { }
8              while( pivot < a[ --j ] ) { }
9              if( i < j )
10                 std::swap( a[ i ], a[ j ] );
11             else
12                 break;
13         }
14         std::swap( a[ i ], a[ right - 1 ] ); // Restore pivot
15         quicksort( a, left, i - 1 );         // Sort S1
16         quicksort( a, i + 1, right );        // Sort S2
17     }
18     else // Size < 10: use insertion sort
19         insertionSort( a, left, right );
20 }
```

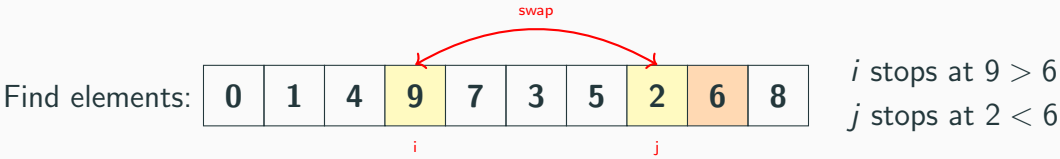
Quicksort Partitioning Trace

Partitioning array segment: $[8, 1, 4, 9, 0, 3, 5, 2, 7, 6]$ (pivot $= 6$, hidden at index 8)



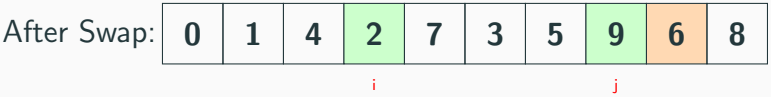
Quicksort Partitioning Trace

Partitioning array segment: $[8, 1, 4, 9, 0, 3, 5, 2, 7, 6]$ (pivot $= 6$, hidden at index 8)



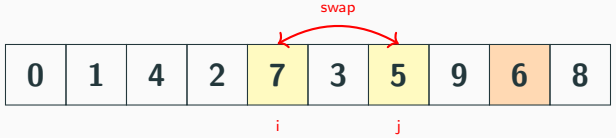
Quicksort Partitioning Trace

Partitioning array segment: $[8, 1, 4, 9, 0, 3, 5, 2, 7, 6]$ (pivot $= 6$, hidden at index 8)



Quicksort Partitioning Trace

Partitioning array segment: $[8, 1, 4, 9, 0, 3, 5, 2, 7, 6]$ (pivot $= 6$, hidden at index 8)



i stops at $7 > 6$

j stops at $5 < 6$

Quicksort Partitioning Trace

Partitioning array segment: $[8, 1, 4, 9, 0, 3, 5, 2, 7, 6]$ (pivot $= 6$, hidden at index 8)



Quicksort Partitioning Trace

Partitioning array segment: $[8, 1, 4, 9, 0, 3, 5, 2, 7, 6]$ (pivot $= 6$, hidden at index 8)

Pointers cross:



i advances to index 6

j recedes to index 5 (

$i > j \implies$ Break!

Quicksort Partitioning Trace

Partitioning array segment: $[8, 1, 4, 9, 0, 3, 5, 2, 7, 6]$ (pivot $= 6$, hidden at index 8)



Quicksort Partitioning Trace

Partitioning array segment: $[8, 1, 4, 9, 0, 3, 5, 2, 7, 6]$ (pivot $= 6$, hidden at index 8)

Done:



Pivot 6 is sorted!

$S_1 \leq 6$ (indices 0..5)

$S_2 \geq 6$ (indices 7..9)

Quicksort Complexity: Worst & Best Cases

Worst Case:

- Pivot is consistently the smallest/largest element.
- Recurrence: $T(N) = T(N-1) + cN$.
- Telescoping:

$$\begin{aligned} T(N) - T(N-1) &= cN \\ T(N-1) - T(N-2) &= c(N-1) \\ &\vdots \\ T(2) - T(1) &= c(2) \end{aligned}$$

Adding all equations: $T(N) = T(1) + c\sum_{i=2}^{N} i = \Theta(N^2)$.

Best Case:

- Pivot is always the exact median.
- Recurrence: $T(N) = 2T(N/2) + cN \implies \Theta(N \log N)$ (same as Mergesort).

Quicksort Complexity: Average Case

Assume each partition size i (from 0 to $N-1$) is equally likely with probability $1/N$:

$$T(N) = \frac{2}{N}\left[\sum_{j=0}^{N-1} T(j)\right] + cN$$

Multiply both sides by N:

$$NT(N) = 2\left[\sum_{j=0}^{N-1} T(j)\right] + cN^2 \quad \text{— (Eq 1)}$$

Write the equation for $N-1$:

$$(N-1)T(N-1) = 2\left[\sum_{j=0}^{N-2} T(j)\right] + c(N-1)^2 \quad \text{— (Eq 2)}$$

Quicksort Complexity: Average Case (Cont.)

Subtract Eq 2 from Eq 1 to eliminate the summation:

$$NT(N) - (N-1)T(N-1) = 2T(N-1) + c(2N-1)$$

Rearrange terms and drop the insignificant $-c$:

$$NT(N) = (N+1)T(N-1) + 2cN$$

Divide both sides by $N(N+1)$ to set up telescoping:

$$\frac{T(N)}{N+1} = \frac{T(N-1)}{N} + \frac{2c}{N+1}$$

Quicksort Complexity: Average Case (Cont.)

Telescope the division relation:

$$\frac{T(N)}{N+1} = \frac{T(1)}{2} + 2c \sum_{i=3}^{N+1} \frac{1}{i}$$

Using the Harmonic number approximation $\sum_{i=1}^{M} \frac{1}{i} \approx \ln M$:

$$\frac{T(N)}{N+1} \approx 2c \ln N$$
$$T(N) \approx 2cN \ln N = \mathcal{O}(N \log N)$$

Stability:

- **Unstable**. Shifting elements past the pivot during partitioning can easily reorder equal elements.

Heapsort

Heapsort: Core Idea

Concept:

- Use a priority queue (Binary Heap) to sort in-place with no extra memory.
- Since we want to sort the array in **ascending** order:
  1. Build a **Max-Heap** of the N elements in-place in $\mathcal{O}(N)$ time.
  2. Repeatedly delete the maximum element:
     - Swap the root element (index 0, the max element) with the last element of the heap (index j).
     - Decrement the logical heap size by 1.
     - Percolate down the new root element to restore the max-heap property.
- After $N - 1$ swaps, the array is sorted in ascending order!

Heapsort vs. Chapter 7 Priority Queue

How does Heapsort's heap implementation compare to standard Priority Queues (Chapter 7)?

Feature	Priority Queue (Chapter 7)	Heapsort (Chapter 10)
Heap Type	Min-Heap (min element at root)	Max-Heap (max element at root)
Sorting Goal	Extract minimum element first	Sort array in ascending order in-place
Indexing	1-based indexing (root at index 1)	0-based indexing (root at index 0)
Left Child	`leftChild(i) = 2 * i`	`leftChild(i) = 2 * i + 1`
Right Child	`rightChild(i) = 2 * i + 1`	`rightChild(i) = 2 * i + 2`
Sentinel	Used `array[0]` as a sentinel	No sentinel used (explicit bounds check)

Heapsort vs. Chapter 7 Priority Queue

Why Max-Heap for Ascending Order?
By using a **Max-Heap**, the largest element is always at index 0. When we swap index 0 with the last element of the heap, the maximum element naturally lands at its correct position at the end of the array. Building a Min-Heap instead would result in a descending sort (unless we use extra memory).

Heapsort C++ Code

```
1  template <typename Comparable>
2  void heapsort( vector<Comparable> & a ) {
3      // Phase 1: buildHeap
4      for( int i = a.size() / 2 - 1; i >= 0; --i )
5          percDown( a, i, a.size() );
6
7      // Phase 2: deleteMax swaps
8      for( int j = a.size() - 1; j > 0; --j ) {
9          std::swap( a[ 0 ], a[ j ] );
10         percDown( a, 0, j );
11     }
12 }
```

- Left child index for element *i* is ‘leftChild(i) = 2*i + 1‘.
- Max-heap structure is stored in-place inside ‘vector a‘.

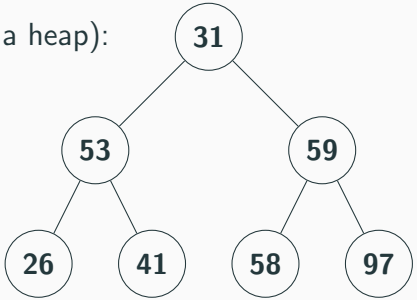
Heapsort: ‘percDown‘ Implementation

```
1  template <typename Comparable>
2  void percDown( vector<Comparable> & a, int i, int n ) {
3      int child;
4      Comparable tmp;
5      for( tmp = std::move( a[ i ] ); 2 * i + 1 < n; i = child ) {
6          child = 2 * i + 1; // Left child
7          if( child != n - 1 && a[ child ] < a[ child + 1 ] )
8              ++child;       // Point to larger child
9          if( tmp < a[ child ] )
10             a[ i ] = std::move( a[ child ] );
11         else
12             break;
13     }
14     a[ i ] = std::move( tmp );
15 }
```

Heapsort Visual Trace: Heap Construction

Build Max-Heap on: $[31, 53, 59, 26, 41, 58, 97]$

Initial (not a heap):

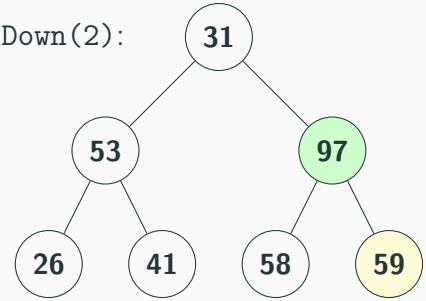


Call `percDown` on index 2 (val 59)

Heapsort Visual Trace: Heap Construction

Build Max-Heap on: $[31, 53, 59, 26, 41, 58, 97]$

After `percDown(2)`:



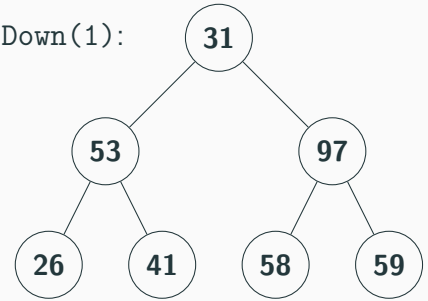
Call `percDown` on index 1 (val 53)

No change (53 ¿ 26, 41)

Heapsort Visual Trace: Heap Construction

Build Max-Heap on: $[31, 53, 59, 26, 41, 58, 97]$

After `percDown(1)`:

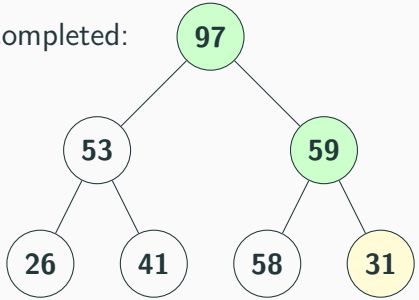


Call `percDown` on index 0 (val 31)

Heapsort Visual Trace: Heap Construction

Build Max-Heap on: $[31, 53, 59, 26, 41, 58, 97]$

Max-Heap Completed:



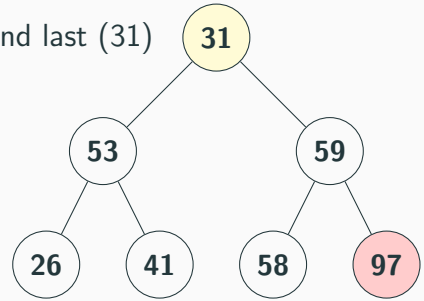
Array matches Figure 7.8:

`97, 53, 59, 26, 41, 58, 31`

Heapsort Visual Trace: Sorting (deleteMax)

Heap: $[97, 53, 59, 26, 41, 58, 31]$ (logical size $= 7$)

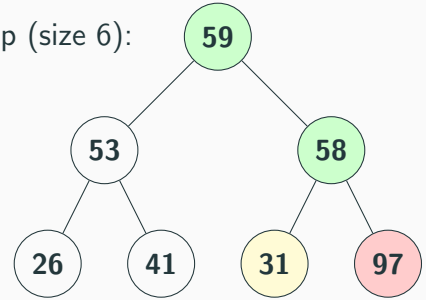
Step 1: swap root (97) and last (31)



Heapsort Visual Trace: Sorting (deleteMax)

Heap: $[97, 53, 59, 26, 41, 58, 31]$ (logical size $= 7$)

Restore heap (size 6):



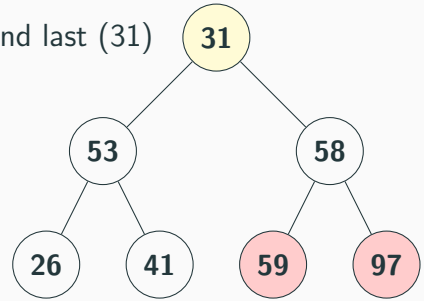
Matches Figure 7.9:

`59, 53, 58, 26, 41, 31 | 97`

Heapsort Visual Trace: Sorting (deleteMax)

Heap: $[97, 53, 59, 26, 41, 58, 31]$ (logical size $= 7$)

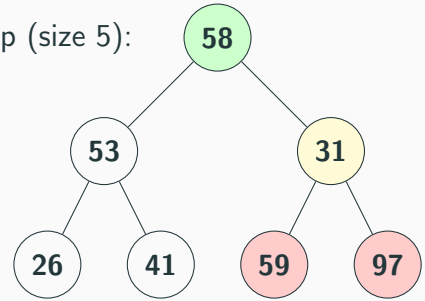
Step 2: swap root (59) and last (31)



Heapsort Visual Trace: Sorting (deleteMax)

Heap: $[97, 53, 59, 26, 41, 58, 31]$ (logical size $= 7$)

Restore heap (size 5):



Heapsort Complexity and Properties

Complexity:

- **Worst, Best, and Average case**: $\Theta(N \log N)$.
- Excellent and highly consistent. Runs in $2N \log N - \mathcal{O}(N \log \log N)$ comparisons on average.

Space Complexity:

- $\mathcal{O}(1)$ auxiliary space (completely in-place).
- *Advantage*: Preferred over Mergesort when space is constrained.

Stability:

- **Unstable**. Long-range swaps during heap operations destroy original relative ordering of equal elements.

Non-Comparison Sorts (Linear Time)

Lower Bound for Comparison Sorts

Theorem: Any comparison-based sorting algorithm requires at least $\lceil \log_2(N!) \rceil$ comparisons in the worst case. **Proof via Decision Trees**:

- A binary tree representing comparisons made by a sorting algorithm on N elements.
- Each internal node represents a comparison $a_i < a_j$.
- Each leaf represents a unique sorted permutation of the input elements.
- Since there are $N!$ possible permutations of N elements, the decision tree must contain at least $N!$ leaves.
- A binary tree of height h can have at most 2^h leaves.

$$2^h \geq N! \implies h \geq \log_2(N!)$$

Lower Bound Proof: Stirling's Approximation

Using Stirling's approximation for $N!$:

$$N! \approx \sqrt{2\pi N} \left(\frac{N}{e}\right)^N$$

Taking the binary logarithm:

$$\begin{aligned}\log_2(N!) &\approx N \log_2 N - N \log_2 e + \frac{1}{2} \log_2(2\pi N) \\ &\geq N \log_2 N - 1.44N \\ &= \Omega(N \log N)\end{aligned}$$

Corollary: Any comparison-based sort has a worst-case lower bound of $\Omega(N \log N)$.

To achieve linear time ($\mathcal{O}(N)$), we must use algorithms that do not rely solely on comparisons!

Bucket Sort

Concept:

- Assume the input elements are integers in the range $[0, M-1]$.
- Maintain an array of M empty lists (called **buckets**).
- Scan the input array and place each element x into bucket x.
- Concatenate all the buckets sequentially.

Complexity:

- Distribution takes $\mathcal{O}(N)$ time.
- Concatenation takes $\mathcal{O}(M)$ time.
- Total Time Complexity: $\mathcal{O}(N+M)$.
- If $M = \mathcal{O}(N)$ (range is small), Bucket Sort runs in **linear time** $\mathcal{O}(N)$!

Counting Sort

Concept:

- A variant of bucket sort that uses an integer array to count occurrences.
- Can be made **stable** using cumulative frequencies.

Algorithm Steps:

1. Count occurrences of each key into ‘count‘ array of size M.
2. Transform ‘count‘ into prefix sums: ‘count[i] += count[i-1]‘.
3. Scan input array **from back to front**:
   - For element x, its sorted index is ‘count[x] - 1‘.
   - Write x to output buffer at this index.
   - Decrement ‘count[x]‘.

Counting Sort Trace

Sort array: $[4, 2, 2, 8, 3, 3, 1]$ (keys range 1 to 8)

Input Array A:

$\mathbf{4}$	2_a	2_b	$\mathbf{8}$	3_a	3_b	$\mathbf{1}$

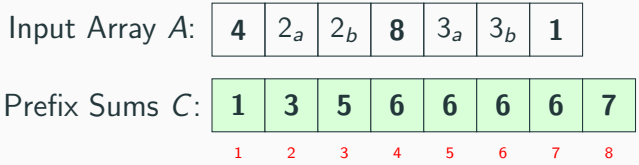
Counts C:

$\mathbf{1}$	$\mathbf{2}$	$\mathbf{2}$	$\mathbf{1}$	$\mathbf{0}$	$\mathbf{0}$	$\mathbf{0}$	$\mathbf{1}$
1	2	3	4	5	6	7	8

Count frequency of each key

Counting Sort Trace

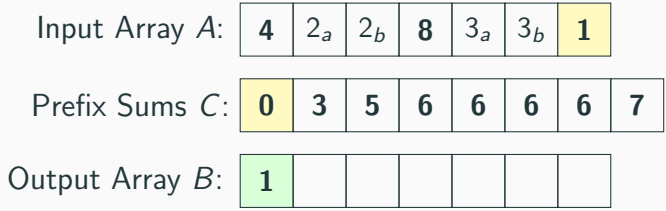
Sort array: $[4, 2, 2, 8, 3, 3, 1]$ (keys range 1 to 8)



Compute cumulative prefix sums

Counting Sort Trace

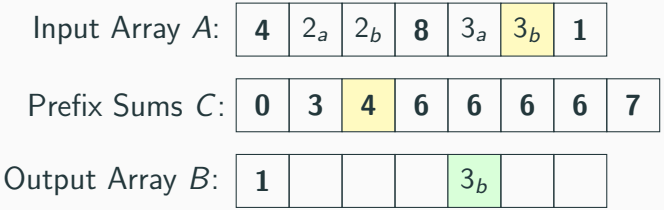
Sort array: $[4, 2, 2, 8, 3, 3, 1]$ (keys range 1 to 8)



Scan 1: $C[1] = 1 \rightarrow B[0] = 1$. D

Counting Sort Trace

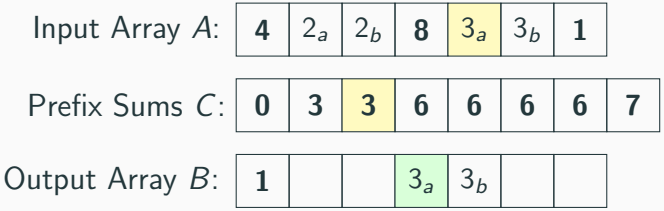
Sort array: $[4, 2, 2, 8, 3, 3, 1]$ (keys range 1 to 8)



Scan 3_b: $C[3] = 5 \rightarrow B[4] = 3_b$.

Counting Sort Trace

Sort array: $[4, 2, 2, 8, 3, 3, 1]$ (keys range 1 to 8)



Scan 3_a: $C[3] = 4 \rightarrow B[3] = 3_a$.

Radix Sort

Concept:

- Sorts integers (or strings) digit-by-digit.
- Must sort from **Least Significant Digit (LSD)** to **Most Significant Digit (MSD)** using a **stable** sorting routine (like counting sort or stable bucket sort).
- If we sort by MSD first, subsequent passes on lower digits will destroy the high-order sorted structure!

Radix Sort

Complexity:

- If keys have P digits (passes) and radix size is B (buckets):

$$\mathcal{O}(P(N+B)) \text{ time complexity}$$

- If P and B are constants, Radix Sort runs in **linear time** $\mathcal{O}(N)$!

Radix Sort Trace

Sort array: $[329, 457, 657, 839, 436, 720, 355]$

Input	Pass 1 (Ones)	Pass 2 (Tens)	Pass 3 (Hundreds)
329	72**0**	7**2**0	**3**29
457	35**5**	3**2**9	**3**55
657	43**6**	4**3**6	**4**36
839	45**7**	8**3**9	**4**57
436	65**7**	3**5**5	**6**57
720	32**9**	4**5**7	**7**20
355	83**9**	6**5**7	**8**39

- Bold digits represent the key used in each pass.
- Notice that equal digits in one pass preserve their relative order from the previous pass (e.g. 457 and 657 have the same ones digit 7, and they remain in the same relative order 457 $\rightarrow$ 657 in Pass 1).

Radix Sort: C++ Implementation

```
1  /* Radix sort an array of Strings of the same length */
2  void radixSortA( vector<string> & arr, int stringLen ) {
3      const int BUCKETS = 256;
4      vector<vector<string>> buckets( BUCKETS );
5
6      // Sort starting from LSD to MSD
7      for( int pos = stringLen - 1; pos >= 0; --pos ) {
8          for( string & s : arr )
9              buckets[ s[ pos ] ].push_back( std::move( s ) );
10
11         int idx = 0;
12         for( auto & thisBucket : buckets ) {
13             for( string & s : thisBucket )
14                 arr[ idx++ ] = std::move( s );
15             thisBucket.clear( );
16         }
17     }
18 }
```

Comparison of Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable?
Selection Sort	N^2	N^2	N^2	1	No
Insertion Sort	N	N^2	N^2	1	Yes
Merge Sort	$N \log N$	$N \log N$	$N \log N$	N	Yes
Quicksort	$N \log N$	$N \log N$	N^2	$\log N$	No
Heapsort	$N \log N$	$N \log N$	$N \log N$	1	No
Radix Sort	$P(N + B)$	$P(N + B)$	$P(N + B)$	$N + B$	Yes

Takeaways:

- Use **Quicksort** for general-purpose high-performance sorting.
- Use **Mergesort** if stability is required and extra memory is available.
- Use **Heapsort** if memory is constrained and stability is not required.